데이터베이스 실무 기말과제

패션 상품 옵션별 재고·주문 관리 데이터베이스 시스템

학과명 인공지능소프트웨어학과

교과목명 데이터베이스 실무

담당교수명 민경주

학번 2601110288

성명 박수용

목차

1. 과제 개요

1-1. 개발 요구사항

1-2. 과제 개요

2. 요구사항분석 명세서

2-1. 기능 요구사항 명세서

2-2. 데이터 관리 요구사항

3. 개념적 설계

3-1. Entity 정의

3-2. 개체 간 관계 정의

3-3. 개념 ERD

4. 논리적 설계

4-1. 논리 ERD

4-2. 릴레이션 스키마

4-3. 테이블 명세서

4-4. 기본키 및 외래키 설계

5. 구현

5-1. Oracle 접속 및 테이블 생성

5-2. 샘플 데이터 입력

5-3. 주문 조회 및 판매 집계

5-4. 재고 차감과 롤백

5-5. 구현 결과

부록. Oracle 구현 SQL

1. 과제 개요

본 과제는 패션 상품의 색상과 사이즈에 따라 달라지는 재고를 관리하고, 회원의 주문과 재고 변동을 함께 기록하는 데이터베이스를 설계하는 과제입니다.

회원, 상품, 주문, 재고 정보를 연결하여 주문 내역과 재고 수량을 일관되게 관리하고자 합니다.

1-1. 개발 요구사항

시스템에서 구현할 주요 기능은 다음과 같습니다. 세부 처리 조건과 확인 기준은 2장의 요구사항분석 명세서에 정리하였습니다.

번호	개발 요구사항
1	회원의 이름과 이메일을 등록하고, 주문 내역에서 주문 회원을 확인할 수 있어야 합니다.
2	제조업체와 상품을 등록하고, 상품에 해당 제조업체를 연결할 수 있어야 합니다.
3	상품별 색상과 사이즈를 구분하여 옵션과 재고 수량을 관리할 수 있어야 합니다.
4	하나의 주문에 여러 상품옵션을 포함하고, 옵션별 수량과 주문단가를 저장할 수 있어야 합니다.
5	정상 주문 시 주문 수량만큼 해당 상품옵션의 재고를 차감해야 합니다.
6	주문한 옵션 중 하나라도 재고가 부족하면 주문 전체를 저장하지 않아야 합니다.
7	회원별 주문 내역과 상품별 판매 수량 및 판매 금액을 조회할 수 있어야 합니다.
8	상품옵션의 입고와 출고 이력을 기록하고, 주문 출고의 원인을 추적할 수 있어야 합니다.

1-2. 과제 개요

1) 주제 선정 배경

패션 상품은 같은 상품명으로 판매되더라도 색상과 사이즈에 따라 실제 보유 수량이 다릅니다. 상품 전체의 재고만 저장하면 검정색 M 사이즈와 흰색 L 사이즈의 재고를 구분하기 어렵습니다. 따라서 상품의 공통 정보와 옵션별 정보를 나누어 관리하는 구조가 필요하다고 생각하였습니다.

이러한 특징을 반영하여 「**패션 상품 옵션별 재고·주문 관리 데이터베이스 시스템**」을 주제로 선정하였습니다. 수업에서 배운 개체와 관계, 기본키와 외래키, 정규화 및 JOIN을 적용하고, 주문 등록과 재고 변경을 함께 처리하는 과정을 구현하는 것이 목적입니다.

2) 설계 및 구현 범위

회원, 제조업체, 상품, 상품옵션, 주문, 주문상세, 재고변동이력의 7개 테이블로 구성하였습니다. 주문과 주문상세를 분리하여 한 주문에 여러 옵션을 담고, 주문 당시의 단가를 보관하도록 하였습니다.

Oracle에서 테이블 생성, 샘플 입력, 입고 및 주문 처리, 판매 집계 SQL을 작성하였습니다. 같은 구조는 이후 Java JDBC에서 주문 등록과 재고 조회를 구현할 때 사용할 수 있습니다.

2. 요구사항분석 명세서

개발 요구사항을 실제 저장 데이터와 처리 결과에 맞추어 구체화하였습니다.

아래 표에는 기능별 요구사항과 기대 결과를 정리하였습니다. 실습 결과는 5장에 정리하였습니다.

2-1. 기능 요구사항 명세서

ID 및 기능	요구사항	구현 후 확인 기준
F-01 회원 관리	회원번호, 이름, 이메일을 등록하고 주문 내역에서 회원 정보를 확인합니다.	주문 내역에 주문한 회원의 이름이 표시되어야 합니다.
F-02 제조업체 및 상품 관리	제조업체와 상품을 등록하고 상품의 제조업체를 연결합니다.	상품과 제조업체를 함께 조회할 수 있어야 하며, 존재하지 않는 제조업체는 참조할 수 없어야 합니다.
F-03 상품옵션 관리	상품별 색상·사이즈 조합을 등록하고 옵션별 재고를 조회합니다. 입고로 재고를 증가시킵니다.	동일 상품의 같은 색상·사이즈는 중복 등록되지 않아야 합니다. 입고 후 재고와 이력이 함께 반영되어야 합니다.
F-04 주문 등록	회원의 주문 일시와 옵션별 수량·주문단가를 저장합니다. 한 주문에서 같은 옵션은 한 행만 저장합니다.	한 주문에 여러 옵션을 저장할 수 있으며, 상품 가격이 바뀌어도 기존 주문단가는 유지되어야 합니다.
F-05 재고 차감	주문이 정상 처리되면 옵션별 재고를 주문 수량만큼 차감하고 출고 이력을 기록합니다.	재고 10개인 옵션을 2개 주문하면 재고가 8개가 되고 해당 주문상세의 출고 이력이 한 건 생성되어야 합니다.
F-06 재고 부족 확인	각 옵션의 재고를 확인하고, 하나라도 부족하면 주문 전체를 실패 처리합니다.	재고 부족 시 해당 주문·주문상세·출고 이력이 남지 않고 모든 옵션의 재고가 처리 전 값으로 유지되어야 합니다.
F-07 주문 및 판매 내역 조회	회원별 주문 목록과 상세 내역, 상품별 판매 수량 및 판매 금액을 조회합니다.	판매 수량은 주문 수량의 합계, 판매 금액은 주문 수량과 주문단가를 곱한 값의 합계와 일치해야 합니다.
F-08 재고변동 이력 관리	옵션별 입고·출고 수량과 시점을 기록합니다. 주문 출고는 원인이 된 주문상세에 연결합니다.	입고는 주문상세 없이 저장되고, 출고는 주문상세와 연결되어야 합니다. 옵션별 이력 합계와 현재 재고가 일치해야 합니다.

2-2. 데이터 관리 요구사항

1) 관리할 데이터

각 데이터는 고유한 번호로 구분하며, 서로 관련된 데이터는 외래키로 연결할 계획입니다.

재고변동이력의 주문상세번호는 입고일 때 비워두고 주문 출고일 때 기록합니다.

데이터	주요 저장 항목
회원	회원번호, 이름, 이메일
제조업체	제조업체번호, 업체명, 연락처
상품	상품번호, 상품명, 가격, 제조업체번호
상품옵션	옵션번호, 상품번호, 색상, 사이즈, 재고 수량
주문	주문번호, 회원번호, 주문 일시
주문상세	주문상세번호, 주문번호, 옵션번호, 주문 수량, 주문단가
재고변동이력	이력번호, 옵션번호, 주문상세번호, 변경 유형, 변경 수량, 변경 일시

2) 데이터 식별과 중복 방지

각 테이블의 식별번호를 기본키로 설정하여 중복과 누락을 허용하지 않을 계획입니다. 같은 상품번호·색상·사이즈 조합은 한 번만 등록하고, 한 주문 안에서 동일 옵션은 주문상세 한 행으로 관리할 계획입니다. 색상과 사이즈는 공백이나 표기 차이로 중복이 생기지 않도록 정한 표기로 입력하고자 합니다.

3) 관계 및 필수 정보 관리

주문은 등록된 회원 한 명에 속하고, 상품은 제조업체 한 곳에 속하도록 설계하였습니다. 상품옵션은 상품 하나에, 주문상세는 주문 하나와 상품옵션 하나에 연결합니다. 이름, 상품명, 색상, 사이즈 등 업무에 필요한 필수값은 누락되지 않도록 관리할 계획입니다.

3) 4) 가격과 수량 관리

상품 가격과 주문단가는 0 이상, 재고 수량은 0 이상의 정수, 주문 수량과 변경 수량은 1 이상의 정수로 제한할 계획입니다. 주문상세의 주문단가는 주문 시점의 상품 가격을 기록하며, 이후 상품 가격 변경이 기존 주문에 영향을 주지 않도록 관리하고자 합니다.

판매 금액은 현재 상품 가격이 아닌 주문상세의 주문 수량과 주문단가를 기준으로 계산할 계획입니다. 이 과제에서는 옵션별 추가 금액, 할인, 배송비를 별도로 계산하지 않습니다.

5) 주문과 재고 처리

정상 저장된 주문에는 주문상세가 한 개 이상 포함되어야 합니다. 주문 저장, 주문상세 저장, 옵션별 재고 차감, 출고 이력 기록을 하나의 트랜잭션으로 처리할 계획입니다. 여러 옵션 중 하나라도 재고가 부족하거나 처리 중 오류가 발생하면 전체 변경 사항을 취소하고자 합니다.

동시에 여러 주문이 들어와도 재고가 음수가 되지 않도록 재고 조건을 포함한 차감 방식 등을 적용할 계획입니다. 재고가 충분한지 조회하는 것만으로 주문 가능 여부를 확정하지 않고, 실제 차감 시점에도 조건을 확인하도록 구현하고자 합니다.

6) 재고변동이력 관리

상품옵션은 재고 0으로 등록하고, 초기 보유 수량도 입고 이력으로 기록할 계획입니다. 변경 유형은 입고와 출고로 구분하며, 변경 수량은 양수로 저장합니다. 현재 재고는 누적 입고 수량에서 누적 출고 수량을 뺀 값과 일치하도록 관리하고자 합니다.

본 과제의 출고는 주문에 따른 재고 차감으로 한정합니다. 분할 출고는 다루지 않으므로 주문상세 하나에 출고 이력 하나를 기록할 계획입니다. 연결된 주문상세와 출고 이력은 옵션과 수량이 일치해야 하며, 같은 주문상세의 출고 이력이 중복 생성되지 않도록 제한하고자 합니다. 입고 시에는 재고 증가와 입고 이력 저장도 함께 처리할 계획입니다.

7) 삭제 및 변경 제한

주문이 참조하는 회원과 상품이 참조하는 제조업체처럼 다른 행이 참조하는 데이터의 삭제는 외래키로 제한하였습니다. 주문과 재고 이력의 직접 수정·삭제 제한은 이후 JDBC에서 적용할 업무 규칙으로 정하였습니다.

8) 구현 후 확인할 주요 상황

확인 상황	기대 결과
정상 주문	주문·상세·출고 이력이 저장되고 재고가 주문 수량만큼 감소
여러 옵션 중 재고 부족	주문 전체 취소, 관련 신규 기록 없음, 모든 옵션 재고 유지
같은 옵션의 동시 주문	재고를 초과한 주문은 거부되고 최종 재고가 음수가 되지 않음
이력 및 참조 무결성	현재 재고와 입출고 합계 일치, 참조 중인 데이터의 삭제 거부

3. 개념적 설계

앞서 정리한 요구사항을 바탕으로 회원, 제조업체, 상품, 상품옵션, 주문, 주문상세, 재고변동이력의 7개 개체를 정의하였습니다. 같은 상품이라도 색상과 사이즈에 따라 재고가 달라지므로 상품과 상품옵션을 분리하였습니다. 또한 하나의 주문에 여러 옵션이 포함될 수 있도록 주문과 주문상세를 구분하였습니다.

3-1. Entity 정의

Entity는 시스템에서 정보를 구분하여 관리하는 대상인 개체를 의미합니다. 각 개체의 주요 속성과 기본 식별자는 다음과 같습니다.

개체	정의 및 주요 속성	기본 식별자
회원	상품을 주문하는 회원의 정보를 관리합니다. 주요 속성: 회원번호, 이름, 이메일	회원번호
제조업체	상품을 제조하는 업체의 정보를 관리합니다. 주요 속성: 제조업체번호, 업체명, 연락처	제조업체번호
상품	판매하는 상품의 공통 정보를 관리합니다. 주요 속성: 상품번호, 상품명, 가격	상품번호
상품옵션	상품의 색상과 사이즈별 재고를 관리합니다. 주요 속성: 옵션번호, 색상, 사이즈, 재고 수량	옵션번호
주문	회원이 주문한 거래 단위와 시점을 관리합니다. 주요 속성: 주문번호, 주문 일시	주문번호
주문상세	주문에 포함된 옵션별 구매 정보를 관리합니다. 주요 속성: 주문상세번호, 주문 수량, 주문단가	주문상세번호
재고변동이력	옵션별 입고와 출고 내역을 관리합니다. 주요 속성: 이력번호, 변경 유형, 변경 수량, 변경 일시	이력번호

1) 개체를 분리한 이유

상품에는 상품명과 가격을 저장하고, 상품옵션에는 색상·사이즈와 재고 수량을 저장하도록 설계하였습니다. 이를 통해 동일한 상품 정보를 옵션마다 반복해서 저장하는 것을 줄이고, 옵션별 재고를 구분하여 관리하고자 하였습니다.

주문상세의 주문단가는 주문 당시의 상품 가격입니다. 상품의 현재 가격과 별도로 기록하여 이후 가격이 변경되어도 과거 주문 금액을 확인할 수 있도록 하였습니다. 재고변동이력은 현재 수량만으로 알기 어려운 입출고 원인과 시점을 확인하기 위해 별도 개체로 정의하였습니다.

이 단계에서는 개체 간 연결을 관계로 표현하였습니다. 제조업체번호, 상품번호 등 관계 연결을 위한 외래키 속성은 4장의 릴레이션 스키마와 테이블 명세서에서 구체화할 계획입니다.

3-2. 개체 간 관계 정의

개체 사이의 관계는 다음과 같이 정리하였습니다. 1:N은 하나의 개체가 여러 개의 상대 개체와 연결될 수 있음을 의미합니다. 주문상세와 재고변동이력의 1:1 관계는 주문에 따른 출고에 적용하였습니다.

개체 간 관계	관계 유형	관계 및 참여 조건
회원 → 주문 관계명: 주문	1:N	회원은 주문이 없거나 여러 건일 수 있습니다. 각 주문은 반드시 회원 한 명에 속합니다.
제조업체 → 상품 관계명: 생산	1:N	제조업체는 등록 상품이 없거나 여러 개일 수 있습니다. 각 상품은 제조업체 한 곳에 속하도록 범위를 정하였습니다.
상품 → 상품옵션 관계명: 구성	1:N	상품은 옵션이 없거나 여러 개일 수 있습니다. 각 옵션은 반드시 상품 하나에 속하며, 주문은 등록된 옵션을 대상으로 합니다.
주문 → 주문상세 관계명: 포함	1:N	정상 저장된 주문은 주문상세를 한 개 이상 포함합니다. 각 주문상세는 반드시 주문 하나에 속합니다.
상품옵션 → 주문상세 관계명: 선택	1:N	옵션은 주문된 적이 없거나 여러 주문상세에 나타날 수 있습니다. 각 주문상세는 옵션 하나를 지정합니다.
상품옵션 → 재고변동이력 관계명: 발생	1:N	옵션은 변동 이력이 없거나 여러 건일 수 있습니다. 각 이력은 반드시 옵션 하나에 속합니다.
주문상세 → 재고변동이력 관계명: 출고 기록	1:1 출고 시	정상 처리된 주문상세마다 출고 이력 하나를 기록합니다. 출고 이력은 주문상세 하나에 연결하며, 입고 이력은 주문상세와 연결하지 않습니다.

1) 주문과 상품옵션의 관계

하나의 주문에 여러 상품옵션이 포함될 수 있고, 한 옵션도 여러 주문에서 구매될 수 있습니다. 주문과 상품옵션의 다대다 관계를 주문상세로 나누어 표현하였습니다. 동일 옵션을 여러 번 선택한 경우 수량을 합치는 처리는 향후 JDBC 입력 단계에서 적용할 계획입니다.

2) 출고 기록 관계의 적용 범위

본 과제에서는 출고를 주문에 따른 재고 차감으로 한정하며 분할 출고는 다루지 않습니다. 이에 따라 주문상세 한 건에 출고 이력 한 건을 대응시켰습니다. 두 기록은 같은 옵션을 가리키고 출고 수량은 주문 수량과 일치해야 합니다. 입고는 주문 없이 발생하므로 모든 재고변동이력이 주문상세에 연결되는 것은 아닙니다.

3-3. 개념 ERD

개체와 관계를 종합하여 다음과 같이 개념 ERD를 작성하였습니다.

사각형은 개체, 타원은 속성, 마름모는 관계를 나타내며, 밑줄은 각 개체의 기본 식별자를 의미합니다.

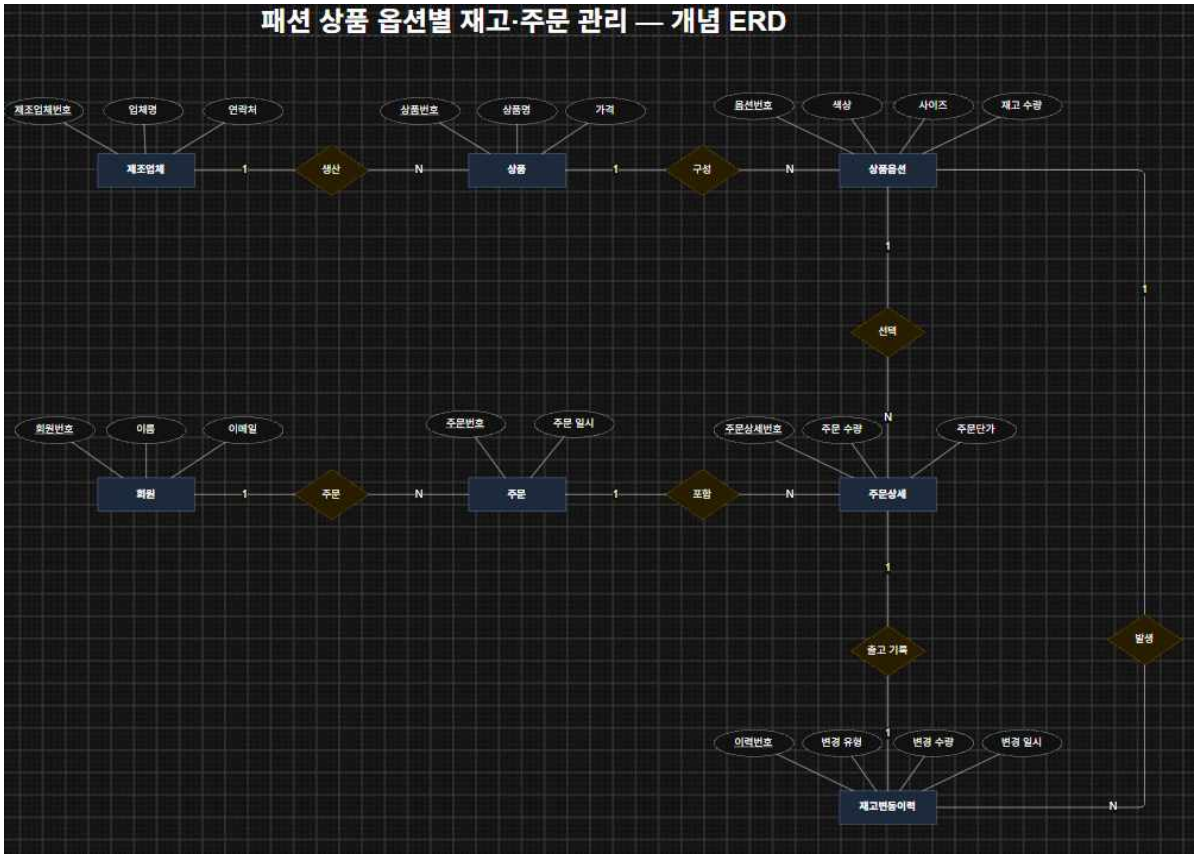


그림 3-1. 패션 상품 옵션별 재고·주문 관리 시스템의 개념 ERD

1) ERD 해석

그림의 1과 N은 관계의 최대 대응 수를 나타냅니다. 회원·제조업체·상품·상품옵션은 관련 하위 기록이 없는 상태도 허용하지만, 정상 저장된 주문에는 주문상세가 한 개 이상 있어야 합니다. 주문상세와 재고변동이력의 출고 기록 관계에서 입고 이력은 참여하지 않습니다.

회원이 검정색 M 사이즈 티셔츠 2개를 주문하면 주문 한 건과 해당 옵션의 주문상세 한 건을 저장할 계획입니다. 재고가 충분하면 해당 옵션의 재고를 2개 차감하고 주문상세와 연결된 출고 이력 한 건을 기록하고자 합니다.

2) 요구사항과의 연결

회원·제조업체·상품 관리는 각 개체로, 옵션별 재고는 상품옵션으로 표현하였습니다. 주문 및 판매 내역은 주문과 주문상세로, 입출고 추적은 재고변동이력으로 반영하였습니다. 옵션 중복 방지와 주문단가 보존 등의 규칙은 이후 논리 설계에서 구체화할 계획입니다.

재고 부족 시 주문 전체 취소와 동시 주문의 안전성은 ERD만으로 보장되지 않습니다. 구현 단계에서 주문 저장, 재고 차감, 출고 이력 기록을 하나의 트랜잭션으로 처리하고, 실제 차감 시점의 재고 조건을 확인할 계획입니다.

4. 논리적 설계

개념적 설계에서 정의한 7개 개체를 테이블로 변환하고, 개체 간 관계를 외래키로 연결하였습니다. 상품과 상품옵션, 주문과 주문상세를 분리한 구조를 유지하면서 실제 저장할 컬럼과 제약조건을 정리하였습니다.

4-1. 논리 ERD

1:N 관계는 N에 해당하는 테이블에 외래키를 배치하였습니다. 주문 출고의 원인을 확인할 수 있도록 재고변동이력에 주문상세번호를 추가하였으며, 입고일 때는 이 값을 비워 두도록 설계하였습니다.

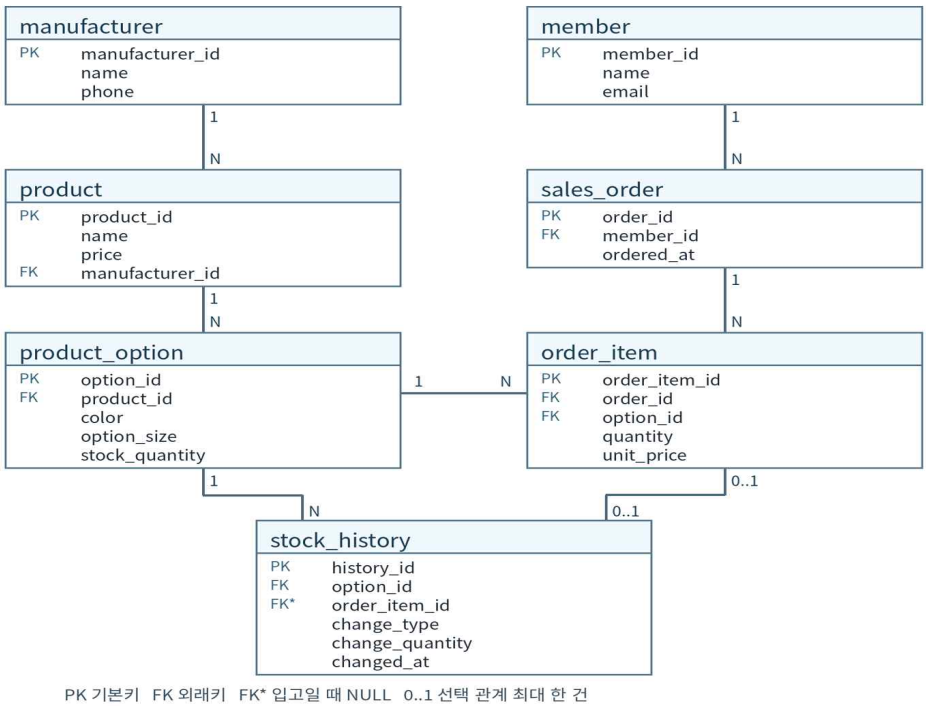


그림 4-1. 기본키와 외래키를 반영한 논리 ERD

정상 주문에는 주문상세가 한 개 이상 포함됩니다. 스키마는 주문상세별 출고 이력을 최대 한 건으로 제한합니다. 정상 주문 처리에서는 각 상세에 출고 이력 한 건을 만들고, 입고 이력은 주문상세와 연결하지 않습니다.

4-2. 릴레이션 스키마

Oracle SQL에 사용한 영문 속성명을 가로로 나열하였습니다. 기본키는 밑줄, 외래키는 파란색 배경으로 표시하였습니다.

회원 member

member_id	name	email

제조업체 manufacturer

manufacturer_id	name	phone

상품 product

product_id	name	price	manufacturer_id

상품옵션 product_option

option_id	product_id	color	option_size	stock_quantity

주문 sales_order

order_id	member_id	ordered_at

주문상세 order_item

order_item_id	order_id	option_id	quantity	unit_price

재고변동이력 stock_history

history_id	option_id	order_item_id	change_type	change_quantity	changed_at

<스키마 구성의 기준>

상품명과 가격, 제조업체는 상품에서 관리하고, 옵션별 색상과 사이즈 및 재고는 상품옵션에서 관리하도록 나누었습니다. 주문의 회원과 일시는 주문에 저장하고, 구매한 옵션과 수량, 주문 당시의 단가는 주문상세에 저장하도록 하였습니다.

주문 합계와 상품별 판매량은 주문상세를 집계하여 계산할 계획입니다. 현재 재고는 상품옵션에 저장하되, 입출고 이력의 합계와 일치하도록 함께 관리하고자 합니다. 이력의 옵션번호와 주문상세가 가리키는 옵션이 어긋나지 않도록 출고 처리 시 확인할 계획입니다.

4-3. 테이블 명세서

Oracle 테이블 생성 SQL을 기준으로 속성 이름, 자료형, 널 허용 여부, 기본값, 기본키, 외래키, 제약조건을 정리하였습니다.

널 허용 여부는 N(허용하지 않음)과 Y(허용함)로 표기하였습니다. UNIQUE 제약은 상품별 색상·사이즈 조합, 주문별 상품옵션, 주문상세별 출고 기록의 중복을 제한합니다.

회원 member

테이블 이름			회원	member		
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
member_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
name	VARCHAR2(50 CHAR)	N				공백만 입력할 수 없음
email	VARCHAR2(254 CHAR)	N				공백만 입력할 수 없음 / 이메일 중복 허용

제조업체 manufacturer

테이블 이름			제조업체	manufacturer		
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
manufacturer_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
name	VARCHAR2(100 CHAR)	N				공백만 입력할 수 없음
phone	VARCHAR2(30 CHAR)	N				CHECK (TRIM(phone) IS NOT NULL)

상품 product

테이블 이름			상품 | product			
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
product_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
name	VARCHAR2(150 CHAR)	N				공백만 입력할 수 없음
price	NUMBER(12,0)	N				CHECK (price ≥ 0)
manufacturer_id	NUMBER(10,0)	N			FK → manufacturer	참조 무결성

연락처는 계산에 사용하는 수치가 아니므로 문자열로 정하였습니다. 금액은 소수점 이하가 없는 원 단위로 저장하며, 주문단가에도 상품 가격과 동일한 자료형을 사용할 계획입니다. 이메일은 로그인 식별자로 사용하지 않으므로 이번 설계에서는 별도 고유 제약을 두지 않았습니다.

상품옵션 product_option

테이블 이름			상품옵션 | product_option			
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
option_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
product_id	NUMBER(10,0)	N			FK → product	참조 무결성
color	VARCHAR2(30 CHAR)	N				공백만 입력할 수 없음
option_size	VARCHAR2(20 CHAR)	N				공백만 입력할 수 없음
stock_quantity	NUMBER(10,0)	N	0			CHECK (stock_quantity ≥ 0)

주문 sales_order

테이블 이름			주문 | sales_order			
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
order_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
member_id	NUMBER(10,0)	N			FK → member	참조 무결성
ordered_at	TIMESTAMP(6) WITH TIME ZONE	N	SYSTIMESTAMP			시간대 정보 포함

주문상세 order_item

테이블 이름			주문상세 | order_item			
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
order_item_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
order_id	NUMBER(10,0)	N			FK → sales_order	UNIQUE(order_id,option_id)
option_id	NUMBER(10,0)	N			FK → product_option	UNIQUE(order_id,option_id)
quantity	NUMBER(10,0)	N				CHECK (quantity ≥ 1)
unit_price	NUMBER(12,0)	N				CHECK (unit_price ≥ 0); 당시 가격

같은 상품의 색상·사이즈 조합과 한 주문 안의 동일 옵션이 중복 저장되지 않도록 복합 UNIQUE 제약을 적용하였습니다.

주문단가는 주문 시점의 가격을 보관하는 값이므로 이후 상품 가격 수정 시 함께 변경하지 않을 계획입니다. 옵션별 추가 금액과 할인, 배송비는 현재 과제 범위에서 제외하였습니다

재고변동이력 stock_history

테이블 이름			재고변동이력 | stock_history			
속성 이름	데이터 타입	널 허용 여부	기본값	기본키	외래키	제약조건
history_id	NUMBER(10,0)	N	—	PK		IDENTITY (1000부터 자동 생성)
option_id	NUMBER(10,0)	N			FK → product_option	참조 무결성
order_item_id	NUMBER(10,0)	Y			FK → order_item	OUT은 필수·유일; IN은 NULL
change_type	VARCHAR2(3 CHAR)	N				IN/OUT 및 주문상세번호 CHECK
change_quantity	NUMBER(10,0)	N				CHECK (change_quantity ≥ 1)
changed_at	TIMESTAMP(6) WITH TIME ZONE	N	SYSTIMESTAMP			변동 발생 시각

4-4. 기본키 및 외래키 설계

각 테이블의 식별번호를 기본키로 사용하고, 참조하는 기본키와 외래키의 자료형을 동일하게 설정하였습니다. 자동 생성 기능과 별도로 PRIMARY KEY를 지정하여 식별번호의 중복을 제한할 계획입니다.

외래키가 있는 테이블	외래키 컬럼	참조 대상
상품	manufacturer_id	manufacturer.manufacturer_id
상품옵션	product_id	product.product_id
주문	member_id	member.member_id
주문상세	order_id	sales_order.order_id
주문상세	option_id	product_option.option_id
재고변동이력	option_id	product_option.option_id
재고변동이력	order_item_id	order_item.order_item_id

참조 중인 데이터의 삭제와 변경

Oracle의 외래키 정의에서 ON DELETE 절을 생략하였습니다. 자식 행이 참조하는 부모 행은 삭제할 수 없으며, 참조 관계가 끊어지는 식별번호 변경도 외래키 제약에 의해 제한됩니다.

제약조건과 주문 처리의 역할

적용 방법	담당할 규칙
PK 및 FK	각 행 식별, 참조하는 데이터의 존재 확인
NOT NULL 및 CHECK	필수값, 수량·가격 범위, 입출고별 주문상세번호 조건
UNIQUE	상품의 색상·사이즈 조합, 주문 안의 동일 옵션, 출고 연결 중복 방지
트랜잭션 및 처리 로직	주문상세 한 개 이상, 상세별 출고 한 건 생성, 옵션·수량 일치, 재고 차감과 이력 저장의 동시 반영

외래키와 UNIQUE만으로 모든 주문상세에 출고 이력이 반드시 존재하는 것은 보장되지 않습니다.

정상 주문 처리에서 필요한 기록을 모두 생성한 뒤 확정하고, 하나라도 실패하면 전체 변경을 취소할 계획입니다.

주문·재고 이력의 변경 제한은 이후 JDBC의 입력과 처리 단계에서 적용할 업무 규칙입니다.

5. 구현

5-1. Oracle 접속 및 테이블 생성

IntelliJ IDEA에서 Oracle의 C##DBSOFT 스키마에 접속하여 7개 테이블을 구성하였습니다. 상품옵션의 사이즈 컬럼은 Oracle 예약어 SIZE를 피하기 위해 OPTION_SIZE로 정의하였습니다. 테이블 생성 SQL은 부록 A에 수록하였습니다.

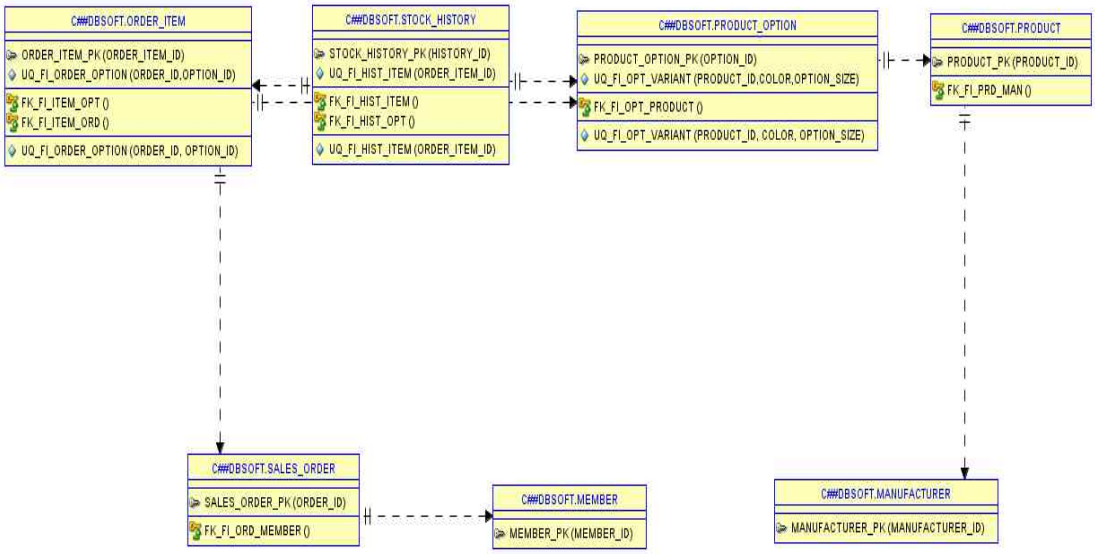


그림 5-1. Oracle Data Modeler에서 확인한 테이블과 제약조건

5-2. 샘플 데이터 입력

제조업체, 회원, 상품을 등록한 뒤 상품옵션 4개를 추가하였습니다. 최초 입고에서는 옵션별 재고 증가와 입고 이력 저장을 한 트랜잭션으로 처리하였습니다. 입력한 수량은 아래와 같으며, 전체 SQL은 부록 B와 C에 수록하였습니다.

옵션번호	상품	색상	사이즈	최초 입고 수량
1	베이직 티셔츠	BLACK	M	10
2	베이직 티셔츠	WHITE	L	5
3	와이드 팬츠	BLACK	M	7
4	와이드 팬츠	BEIGE	L	3

5-3. 주문 조회 및 판매 집계

회원, 주문, 주문상세, 상품옵션, 상품을 JOIN하여 주문 내역을 조회하도록 하였습니다. 판매 기록이 있는 상품은 주문 수량과 주문 당시 단가를 기준으로 판매 수량과 금액을 집계하도록 작성하였습니다. 전체 조회 SQL은 부록 F에 수록하였습니다.

코드	설명	역할
JOIN order_item oi ON oi.order_id = so.order_id	주문번호로 주문과 상세를 연결합니다.	주문에 포함된 상품옵션 조회
SUM(oi.quantity * oi.unit_price)	상세별 수량과 주문단가의 곱을 합산합니다.	판매 금액 계산
GROUP BY p.product_id, p.name	상품번호와 상품명으로 묶습니다.	상품별 판매 수량과 금액 집계

5-4. 재고 차감과 롤백

정상 주문 예제는 옵션 1에서 2개, 옵션 2에서 1개를 차감합니다. 차감 SQL에 재고 조건을 넣어 요청 수량을 충족하는 행만 갱신하도록 하였습니다.

정상 주문의 재고 차감 예시

```
UPDATE product_option
   SET stock_quantity = stock_quantity - 2
 WHERE option_id = 1 AND stock_quantity >= 2;
```

정상 주문 처리 SQL은 부록 D, 재고 부족 시험 SQL은 부록 E에 수록하였습니다.

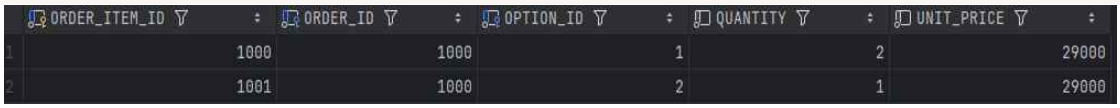
코드	설명	역할
SQL%ROWCOUNT	차감된 행이 1건인지 확인합니다.	차감 실패를 예외로 처리
COMMIT / ROLLBACK	정상 주문은 모든 처리가 끝난 뒤 확정합니다. 예외가 발생하면 주문 처리를 되돌립니다.	주문·상세·재고·이력 일괄 처리
ROLLBACK TO before_expected_failure	재고 부족 시험에서 저장점 이후의 변경을 되돌립니다.	시험 전 상태 복원

재고 부족 시험은 옵션 1의 재고가 1개 이상이고 옵션 2가 존재하며 재고가 999개 미만인 경우를 대상으로 작성하였습니다. 옵션 1을 먼저 차감한 뒤 옵션 2의 차감이 실패하면, 시험 중 생성한 주문·주문상세·출고 이력과 재고 변경을 저장점까지 되돌리도록 하였습니다.

5-5. 구현 결과

IntelliJ의 ORDER_ITEM 조회 화면에서 주문번호 1000에 해당하는 주문상세 두 건을 확인하였습니다. 아래 SQL은 같은 주문만 골라 확인하는 조회문입니다.

```
SELECT order_item_id, order_id, option_id, quantity, unit_price
FROM order_item
WHERE order_id = 1000
ORDER BY order_item_id;
```



	ORDER_ITEM_ID	ORDER_ID	OPTION_ID	QUANTITY	UNIT_PRICE
1	1000	1000	1	2	29000
2	1001	1000	2	1	29000

그림 5-2. IntelliJ에서 확인한 주문 1000의 주문상세

ORDER_ITEM_ID	ORDER_ID	OPTION_ID	QUANTITY	UNIT_PRICE	금액(원)
1000	1000	1	2	29,000	58,000
1001	1000	2	1	29,000	29,000

확인한 주문 수량은 총 3개입니다. 화면에 표시된 수량과 단가를 기준으로 계산한 판매 금액은 87,000원입니다.

실습 결과와 검증 범위

확인 항목	값 또는 기준	근거
테이블 구조	7개 테이블과 OPTION_SIZE	Data Modeler 화면 및 실습 DDL
주문 1000	주문상세 2건 / 총수량 3개 / 87,000원	상세는 IntelliJ 캡처, 합계는 캡처의 수량과 단가로 계산
재고 및 입출고	옵션 1~4 재고 8·4·7·3개 / 입고 4건 / 출고 2건	샘플과 정상 주문을 한 번씩 처리한 경우의 예상값
재고 정합성	현재 재고 = 입고량 합계 − 출고량 합계	부록 F에 비교 SQL 수록
재고 부족 시험	주문·상세·출고·재고를 시험 전 상태로 복원	부록 E에 시험 SQL 수록. 실행 전후 화면 미첨부

이번 과제에서는 옵션별 재고와 주문단가를 분리해 저장하고, 주문 처리 중 오류가 발생했을 때 관련 변경을 함께 취소하도록 SQL을 구성하였습니다.

부록의 조회문은 재고와 입출고 이력의 일치 여부를 확인할 때 사용할 수 있습니다.

부록 A. 02 Oracle 테이블 7개 생성

참조 대상 테이블부터 생성하였습니다.

02~07의 전체 구현문과 개별 조회문을 실행 순서로 정리하였습니다.

생성·입고·정상 주문은 샘플 초기 상태를 기준으로 1회 실행합니다.

```
CREATE TABLE manufacturer (
    manufacturer_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH
1000) PRIMARY KEY,
    name VARCHAR2(100 CHAR) NOT NULL,
    phone VARCHAR2(30 CHAR) NOT NULL,
    CONSTRAINT ck_fi_man_name CHECK (TRIM(name) IS NOT NULL),
    CONSTRAINT ck_fi_man_phone CHECK (TRIM(phone) IS NOT NULL)
);

CREATE TABLE member (
    member_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    name VARCHAR2(50 CHAR) NOT NULL,
    email VARCHAR2(254 CHAR) NOT NULL,
    CONSTRAINT ck_fi_mem_name CHECK (TRIM(name) IS NOT NULL),
    CONSTRAINT ck_fi_mem_email CHECK (TRIM(email) IS NOT NULL)
);

CREATE TABLE product (
    product_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    name VARCHAR2(150 CHAR) NOT NULL,
    price NUMBER(12,0) NOT NULL,
    manufacturer_id NUMBER(10,0) NOT NULL,
    CONSTRAINT ck_fi_prd_name CHECK (TRIM(name) IS NOT NULL),
    CONSTRAINT ck_fi_prd_price CHECK (price >= 0),
    CONSTRAINT fk_fi_prd_man FOREIGN KEY (manufacturer_id)
        REFERENCES manufacturer(manufacturer_id)
);
```

부록 A. 상품옵션·주문·주문상세

```
CREATE TABLE product_option (
    option_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    product_id NUMBER(10,0) NOT NULL,
    color VARCHAR2(30 CHAR) NOT NULL,
    option_size VARCHAR2(20 CHAR) NOT NULL,
    stock_quantity NUMBER(10,0) DEFAULT 0 NOT NULL,
    CONSTRAINT ck_fi_opt_color CHECK (TRIM(color) IS NOT NULL),
    CONSTRAINT ck_fi_opt_size CHECK (TRIM(option_size) IS NOT NULL),
    CONSTRAINT ck_fi_opt_stock CHECK (stock_quantity >= 0),
    CONSTRAINT uq_fi_opt_variant UNIQUE (product_id, color, option_size),
    CONSTRAINT fk_fi_opt_product FOREIGN KEY (product_id)
        REFERENCES product(product_id)
);

CREATE TABLE sales_order (
    order_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    member_id NUMBER(10,0) NOT NULL,
    ordered_at TIMESTAMP(6) WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_fi_ord_member FOREIGN KEY (member_id)
        REFERENCES member(member_id)
);

CREATE TABLE order_item (
    order_item_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    order_id NUMBER(10,0) NOT NULL,
    option_id NUMBER(10,0) NOT NULL,
    quantity NUMBER(10,0) NOT NULL,
    unit_price NUMBER(12,0) NOT NULL,
    CONSTRAINT ck_fi_item_qty CHECK (quantity >= 1),
    CONSTRAINT ck_fi_item_price CHECK (unit_price >= 0),
    CONSTRAINT uq_fi_order_option UNIQUE (order_id, option_id),
    CONSTRAINT fk_fi_item_ord FOREIGN KEY (order_id)
        REFERENCES sales_order(order_id),
    CONSTRAINT fk_fi_item_opt FOREIGN KEY (option_id)
        REFERENCES product_option(option_id)
);
```

부록 A. 재고변동이력

```
CREATE TABLE stock_history (
    history_id NUMBER(10,0) GENERATED BY DEFAULT AS IDENTITY (START WITH 1000)
PRIMARY KEY,
    option_id NUMBER(10,0) NOT NULL,
    order_item_id NUMBER(10,0),
    change_type VARCHAR2(3 CHAR) NOT NULL,
    change_quantity NUMBER(10,0) NOT NULL,
    changed_at TIMESTAMP(6) WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT ck_fi_hist_qty CHECK (change_quantity >= 1),
    CONSTRAINT ck_fi_hist_type CHECK (
      (change_type = 'IN' AND order_item_id IS NULL) OR
      (change_type = 'OUT' AND order_item_id IS NOT NULL)
    ),
    CONSTRAINT uq_fi_hist_item UNIQUE (order_item_id),
    CONSTRAINT fk_fi_hist_opt FOREIGN KEY (option_id)
        REFERENCES product_option(option_id),
    CONSTRAINT fk_fi_hist_item FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
);
```

부록 B. 03 샘플 데이터 입력

제조업체·회원·상품·상품옵션 순으로 샘플을 등록하였습니다.

```
BEGIN
    INSERT INTO manufacturer (manufacturer_id, name, phone)
    VALUES (1, '한빛패션', '02-1234-5678');
    INSERT INTO manufacturer (manufacturer_id, name, phone)
    VALUES (2, '다온의류', '031-234-5678');

    INSERT INTO member (member_id, name, email)
    VALUES (1, '김민지', 'minji@example.com');
    INSERT INTO member (member_id, name, email)
    VALUES (2, '이준호', 'junho@example.com');

    INSERT INTO product (product_id, name, price, manufacturer_id)
    VALUES (1, '베이직 티셔츠', 29000, 1);
    INSERT INTO product (product_id, name, price, manufacturer_id)
    VALUES (2, '와이드 팬츠', 49000, 2);

    INSERT INTO product_option (option_id, product_id, color, option_size)
    VALUES (1, 1, 'BLACK', 'M');
    INSERT INTO product_option (option_id, product_id, color, option_size)
    VALUES (2, 1, 'WHITE', 'L');
    INSERT INTO product_option (option_id, product_id, color, option_size)
    VALUES (3, 2, 'BLACK', 'M');
    INSERT INTO product_option (option_id, product_id, color, option_size)
    VALUES (4, 2, 'BEIGE', 'L');
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
```

부록 C. 04 최초 입고

옵션별 재고 증가와 입고 이력을 함께 기록합니다.

```
BEGIN
    UPDATE product_option SET stock_quantity = stock_quantity + 10 WHERE option_id = 1;
    IF SQL%ROWCOUNT <> 1 THEN RAISE_APPLICATION_ERROR(-20101, '옵션 1 없음'); END
IF;
    INSERT INTO stock_history (option_id, change_type, change_quantity) VALUES (1, 'IN', 10);

    UPDATE product_option SET stock_quantity = stock_quantity + 5 WHERE option_id = 2;
    IF SQL%ROWCOUNT <> 1 THEN RAISE_APPLICATION_ERROR(-20102, '옵션 2 없음'); END
IF;
    INSERT INTO stock_history (option_id, change_type, change_quantity) VALUES (2, 'IN', 5);

    UPDATE product_option SET stock_quantity = stock_quantity + 7 WHERE option_id = 3;
    IF SQL%ROWCOUNT <> 1 THEN RAISE_APPLICATION_ERROR(-20103, '옵션 3 없음'); END
IF;
    INSERT INTO stock_history (option_id, change_type, change_quantity) VALUES (3, 'IN', 7);

    UPDATE product_option SET stock_quantity = stock_quantity + 3 WHERE option_id = 4;
    IF SQL%ROWCOUNT <> 1 THEN RAISE_APPLICATION_ERROR(-20104, '옵션 4 없음'); END
IF;
    INSERT INTO stock_history (option_id, change_type, change_quantity) VALUES (4, 'IN', 3);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
```

부록 D. 05 정상 주문

주문·상세 저장, 재고 차감, 출고 이력을 하나의 트랜잭션으로 처리합니다.

```
DECLARE
    v_order_id sales_order.order_id%TYPE;
    v_item_id order_item.order_item_id%TYPE;
    v_price product.price%TYPE;
BEGIN
    INSERT INTO sales_order (member_id) VALUES (1)
    RETURNING order_id INTO v_order_id;

    -- 첫 번째 옵션: 2개 차감
    SELECT p.price INTO v_price
    FROM product_option po JOIN product p ON p.product_id = po.product_id
    WHERE po.option_id = 1;
    UPDATE product_option
       SET stock_quantity = stock_quantity - 2
     WHERE option_id = 1 AND stock_quantity >= 2;
    IF SQL%ROWCOUNT <> 1 THEN
        RAISE_APPLICATION_ERROR(-20201, '첫 번째 옵션 재고 부족');
    END IF;
    INSERT INTO order_item (order_id, option_id, quantity, unit_price)
    VALUES (v_order_id, 1, 2, v_price)
    RETURNING order_item_id INTO v_item_id;
    INSERT INTO stock_history (option_id, order_item_id, change_type, change_quantity)
    VALUES (1, v_item_id, 'OUT', 2);

    -- 두 번째 옵션: 1개 차감
    SELECT p.price INTO v_price
    FROM product_option po JOIN product p ON p.product_id = po.product_id
    WHERE po.option_id = 2;
    UPDATE product_option
       SET stock_quantity = stock_quantity - 1
     WHERE option_id = 2 AND stock_quantity >= 1;
    IF SQL%ROWCOUNT <> 1 THEN
        RAISE_APPLICATION_ERROR(-20202, '두 번째 옵션 재고 부족');
    END IF;
    INSERT INTO order_item (order_id, option_id, quantity, unit_price)
    VALUES (v_order_id, 2, 1, v_price)
    RETURNING order_item_id INTO v_item_id;
    INSERT INTO stock_history (option_id, order_item_id, change_type, change_quantity)
    VALUES (2, v_item_id, 'OUT', 1);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
```

부록 E. 06 재고 부족 롤백 시험

첫 옵션 차감 이후 두 번째 옵션의 재고 부족을 시험합니다.

```
DECLARE
    v_order_id sales_order.order_id%TYPE;
    v_item_id order_item.order_item_id%TYPE;
    v_price product.price%TYPE;
BEGIN
    SAVEPOINT before_expected_failure;
    INSERT INTO sales_order (member_id) VALUES (1)
    RETURNING order_id INTO v_order_id;

    SELECT p.price INTO v_price
    FROM product_option po JOIN product p ON p.product_id = po.product_id
    WHERE po.option_id = 1;
    UPDATE product_option
       SET stock_quantity = stock_quantity - 1
     WHERE option_id = 1 AND stock_quantity >= 1;
    IF SQL%ROWCOUNT <> 1 THEN
        RAISE_APPLICATION_ERROR(-20302, '예상치 못한 첫 번째 옵션 부족');
    END IF;
    INSERT INTO order_item (order_id, option_id, quantity, unit_price)
    VALUES (v_order_id, 1, 1, v_price)
    RETURNING order_item_id INTO v_item_id;
    INSERT INTO stock_history (option_id, order_item_id, change_type, change_quantity)
    VALUES (1, v_item_id, 'OUT', 1);

    -- 옵션 2의 실제 재고보다 많은 999개를 요청
    UPDATE product_option
       SET stock_quantity = stock_quantity - 999
     WHERE option_id = 2 AND stock_quantity >= 999;
    IF SQL%ROWCOUNT <> 1 THEN
        RAISE_APPLICATION_ERROR(-20301, '예상된 재고 부족');
    END IF;
    -- 차감에 성공하면 시험 변경을 되돌리고 시험 조건 불충족으로 처리합니다.
    ROLLBACK TO before_expected_failure;
    RAISE_APPLICATION_ERROR(-20303, 'FAIL: 재고 부족 조건이 재현되지 않았습니다.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20301 THEN
            ROLLBACK TO before_expected_failure;
            DBMS_OUTPUT.PUT_LINE('PASS: 재고 부족 확인 및 시험 변경사항 복원');
        ELSE
            ROLLBACK TO before_expected_failure;
            RAISE;
        END IF;
END;
```

부록 F. 07 조회 및 정합성 검사

현재 재고, 주문 내역, 판매 금액과 입출고 이력을 조회합니다.

```
-- A. 7개 테이블 실제 존재 확인
SELECT table_name
FROM user_tables
WHERE table_name IN ('MEMBER','MANUFACTURER','PRODUCT','PRODUCT_OPTION',
                     'SALES_ORDER','ORDER_ITEM','STOCK_HISTORY')
ORDER BY table_name;

-- B. 샘플 데이터의 재고 (8, 4, 7, 3이어야 함)
SELECT po.option_id, p.name AS product_name, po.color, po.option_size,
       po.stock_quantity, m.name AS manufacturer_name
FROM product_option po
JOIN product p ON p.product_id = po.product_id
JOIN manufacturer m ON m.manufacturer_id = p.manufacturer_id
ORDER BY po.option_id;

-- C. 회원별 주문과 주문 당시 가격 (2행 예상)
SELECT m.name AS member_name, so.order_id, oi.order_item_id,
       p.name AS product_name, po.color, po.option_size,
       oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_amount
FROM member m
JOIN sales_order so ON so.member_id = m.member_id
JOIN order_item oi ON oi.order_id = so.order_id
JOIN product_option po ON po.option_id = oi.option_id
JOIN product p ON p.product_id = po.product_id
ORDER BY so.order_id, oi.order_item_id;

-- D. 상품별 판매 집계 (티셔츠 3개, 총 87000원)
SELECT p.product_id, p.name AS product_name,
       SUM(oi.quantity) AS sold_quantity,
       SUM(oi.quantity * oi.unit_price) AS sales_amount
FROM product p
JOIN product_option po ON po.product_id = p.product_id
JOIN order_item oi ON oi.option_id = po.option_id
GROUP BY p.product_id, p.name
ORDER BY p.product_id;
```

부록 F. 입출고 및 재고 검증

```
-- E. 입출고 이력 합계와 현재 재고가 같으면 PASS
SELECT po.option_id, po.stock_quantity,
       NVL(SUM(CASE sh.change_type WHEN 'IN' THEN sh.change_quantity
                                   ELSE -sh.change_quantity END), 0) AS calculated_stock,
       CASE WHEN po.stock_quantity =
         NVL(SUM(CASE sh.change_type WHEN 'IN' THEN sh.change_quantity
                                     ELSE -sh.change_quantity END), 0)
       THEN 'PASS' ELSE 'FAIL' END AS result
FROM product_option po
LEFT JOIN stock_history sh ON sh.option_id = po.option_id
GROUP BY po.option_id, po.stock_quantity
ORDER BY po.option_id;

-- F. 주문 수 / 주문상세 수 / 출고 수 / 입고 수: 1/2/2/4
SELECT (SELECT COUNT(*) FROM sales_order) AS order_count,
       (SELECT COUNT(*) FROM order_item) AS order_item_count,
       (SELECT COUNT(*) FROM stock_history WHERE change_type='OUT') AS out_count,
       (SELECT COUNT(*) FROM stock_history WHERE change_type='IN') AS in_count
FROM dual;

-- G. 연결된 출고 이력의 옵션·수량 불일치 확인 (기대: 0행)
SELECT sh.history_id, sh.option_id AS history_option,
       oi.option_id AS item_option, sh.change_quantity, oi.quantity
FROM stock_history sh
JOIN order_item oi ON oi.order_item_id = sh.order_item_id
WHERE sh.change_type='OUT'
  AND (sh.option_id <> oi.option_id OR sh.change_quantity <> oi.quantity);

-- H. 제약조건 상태: 모든 항목 ENABLED / VALIDATED인지 확인
SELECT table_name, constraint_name, constraint_type, status, validated
FROM user_constraints
WHERE table_name IN ('MEMBER','MANUFACTURER','PRODUCT','PRODUCT_OPTION',
                     'SALES_ORDER','ORDER_ITEM','STOCK_HISTORY')
ORDER BY table_name, constraint_type, constraint_name;
```